

Prompt for both AIs:

Create a C program that will prioritize emails for a busy CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritization for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules

- Test File format:

Command	Description
EMAIL <sender category>, <subject line>, <date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two

NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it)

- **Sample Test File:**

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

- **Your program should display this:**

```
There are 6 emails to read.

Next email:
    Sender: Boss
    Subject: Never Mind
    Date: 01-03-2025

Next email:
    Sender: Boss
    Subject: Important
    Date: 12-20-2024

There are 4 emails to read.
```

As you know, the test Case A program. The analysis must include...

Analysis:

2. Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs

I used ChatGPT and Claude.

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed both ChatGPT and Claude on my computer web browser.

c. Prompt used. (You must use the same prompt for both GenAIs.)

Prompt used given above - pictures were given as well

d. Generated code from both GenAIs.

Generated code from ChatGPT:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define LINE_SIZE 2048
#define SENDER_SIZE 32
#define SUBJECT_SIZE 1024
#define DATE_SIZE 11

typedef struct {
    char sender[SENDER_SIZE];
    char subject[SUBJECT_SIZE];
    char date[DATE_SIZE];
    int sender_priority;
    int date_priority;
    unsigned long arrival_order;
} Email;

/* A max heap stored in a dynamically growing list (array). */
typedef struct {
    Email *list;
    size_t size;
    size_t capacity;
} MaxHeap;

static void initializeHeap(MaxHeap *heap)
{
    heap->list = NULL;
```

```

    heap->size = 0;
    heap->capacity = 0;
}

static void destroyHeap(MaxHeap *heap)
{
    free(heap->list);
    heap->list = NULL;
    heap->size = 0;
    heap->capacity = 0;
}

static int getSenderPriority(const char *sender)
{
    if (strcmp(sender, "Boss") == 0) {
        return 5;
    }
    if (strcmp(sender, "Subordinate") == 0) {
        return 4;
    }
    if (strcmp(sender, "Peer") == 0) {
        return 3;
    }
    if (strcmp(sender, "ImportantPerson") == 0) {
        return 2;
    }
    if (strcmp(sender, "OtherPerson") == 0) {
        return 1;
    }
    return 0;
}

/* Converts MM-DD-YYYY to YYYYMMDD so newer dates have larger values. */
static int getDatePriority(const char *date)
{
    int month;
    int day;
    int year;

    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3) {
        return 0;
    }
}

```

```

    return year * 10000 + month * 100 + day;
}

/* Returns nonzero when a has greater priority than b. */
static int hasHigherPriority(const Email *a, const Email *b)
{
    if (a->sender_priority != b->sender_priority) {
        return a->sender_priority > b->sender_priority;
    }
    if (a->date_priority != b->date_priority) {
        return a->date_priority > b->date_priority;
    }

    /* If sender category and date tie, preserve file order. */
    return a->arrival_order < b->arrival_order;
}

static void swapEmails(Email *a, Email *b)
{
    Email temporary = *a;
    *a = *b;
    *b = temporary;
}

static int growHeap(MaxHeap *heap)
{
    size_t new_capacity = (heap->capacity == 0) ? 8 : heap->capacity * 2;
    Email *new_list = realloc(heap->list, new_capacity * sizeof(Email));

    if (new_list == NULL) {
        return 0;
    }

    heap->list = new_list;
    heap->capacity = new_capacity;
    return 1;
}

static void heapifyUp(MaxHeap *heap, size_t index)
{
    while (index > 0) {

```

```

        size_t parent = (index - 1) / 2;

        if (!hasHigherPriority(&heap->list[index], &heap->list[parent])) {
            break;
        }

        swapEmails(&heap->list[index], &heap->list[parent]);
        index = parent;
    }
}

static void heapifyDown(MaxHeap *heap, size_t index)
{
    while (1) {
        size_t left = 2 * index + 1;
        size_t right = 2 * index + 2;
        size_t largest = index;

        if (left < heap->size &&
            hasHigherPriority(&heap->list[left], &heap->list[largest])) {
            largest = left;
        }

        if (right < heap->size &&
            hasHigherPriority(&heap->list[right], &heap->list[largest])) {
            largest = right;
        }

        if (largest == index) {
            break;
        }

        swapEmails(&heap->list[index], &heap->list[largest]);
        index = largest;
    }
}

static int insertEmail(MaxHeap *heap, Email email)
{
    if (heap->size == heap->capacity && !growHeap(heap)) {
        return 0;
    }

    heap->list[heap->size] = email;

```

```

    heapifyUp(heap, heap->size);
    heap->size++;
    return 1;
}

static const Email *peekMax(const MaxHeap *heap)
{
    if (heap->size == 0) {
        return NULL;
    }
    return &heap->list[0];
}

static int removeMax(MaxHeap *heap)
{
    if (heap->size == 0) {
        return 0;
    }

    heap->size--;
    if (heap->size > 0) {
        heap->list[0] = heap->list[heap->size];
        heapifyDown(heap, 0);
    }
    return 1;
}

static void removeLineEnding(char *text)
{
    text[strcspn(text, "\r\n")] = '\0';
}

static int parseEmailLine(char *line, Email *email,
                          unsigned long arrival_order)
{
    char *sender;
    char *subject;
    char *date;

    if (strncmp(line, "EMAIL ", 6) != 0) {
        return 0;
    }

```

```

    sender = line + 6;
    subject = strchr(sender, ',');
    if (subject == NULL) {
        return 0;
    }
    *subject = '\0';
    subject++;

    date = strrchr(subject, ',');
    if (date == NULL) {
        return 0;
    }
    *date = '\0';
    date++;
    removeLineEnding(date);

    if (strlen(sender) >= SENDER_SIZE || strlen(subject) >= SUBJECT_SIZE ||
        strlen(date) >= DATE_SIZE) {
        return 0;
    }

    strcpy(email->sender, sender);
    strcpy(email->subject, subject);
    strcpy(email->date, date);
    email->sender_priority = getSenderPriority(sender);
    email->date_priority = getDatePriority(date);
    email->arrival_order = arrival_order;

    return email->sender_priority != 0 && email->date_priority != 0;
}

static void displayNextEmail(const MaxHeap *heap)
{
    const Email *email = peekMax(heap);

    if (email == NULL) {
        printf("No emails to read.\n\n");
        return;
    }

    printf("Next email:\n");

```

```

    printf("    Sender: %s\n", email->sender);
    printf("    Subject: %s\n", email->subject);
    printf("    Date: %s\n\n", email->date);
}

static void displayCount(const MaxHeap *heap)
{
    printf("There are %zu emails to read.\n\n", heap->size);
}

int main(int argc, char *argv[])
{
    FILE *input = stdin;
    MaxHeap inbox;
    char line[LINE_SIZE];
    unsigned long arrival_order = 0;

    if (argc > 2) {
        fprintf(stderr, "Usage: %s [test_file]\n", argv[0]);
        return EXIT_FAILURE;
    }

    if (argc == 2) {
        input = fopen(argv[1], "r");
        if (input == NULL) {
            fprintf(stderr, "Could not open %s\n", argv[1]);
            return EXIT_FAILURE;
        }
    }

    initializeHeap(&inbox);

    while (fgets(line, sizeof(line), input) != NULL) {
        Email email;
        removeLineEnding(line);

        if (strncmp(line, "EMAIL ", 6) == 0) {
            if (!parseEmailLine(line, &email, arrival_order++)) {
                fprintf(stderr, "Invalid email line.\n");
                destroyHeap(&inbox);
                if (input != stdin) {
                    fclose(input);
                }
            }
        }
    }
}

```

```

        }
        return EXIT_FAILURE;
    }
    if (!insertEmail(&inbox, email)) {
        fprintf(stderr, "Not enough memory.\n");
        destroyHeap(&inbox);
        if (input != stdin) {
            fclose(input);
        }
        return EXIT_FAILURE;
    }
} else if (strcmp(line, "NEXT") == 0) {
    displayNextEmail(&inbox);
} else if (strcmp(line, "READ") == 0) {
    removeMax(&inbox);
} else if (strcmp(line, "COUNT") == 0) {
    displayCount(&inbox);
}
}

destroyHeap(&inbox);
if (input != stdin) {
    fclose(input);
}

return EXIT_SUCCESS;
}

```

Generated code from Claude:

```

/* =====
 * email_priority.c
 *
 * Prioritizes a CEO's inbox using a MaxHeap-based priority queue.
 *
 * The MaxHeap is implemented from scratch as a dynamically-growing
 * array (a "list-based" heap, as opposed to a pointer/tree-based
 * heap). No standard-library or third-party heap module is used --
 * every heap operation below (insert, extract-max, sift up/down)
 * is written by hand.
 *
 */

```

```

* -----
* Priority rules
* -----
* 1. Sender category determines the primary priority:
*     Boss                (highest / read first)
*     Subordinate
*     Peer
*     ImportantPerson
*     OtherPerson         (lowest / read last)
*
* 2. When two queued emails share the same category, the NEWER
*     email (by date) is read first.
*
* Both rules are folded into a single numeric key so a standard
* MaxHeap comparison (bigger key = higher priority) handles both
* at once:
*
*     key = categoryRank * 100000000 + YYYYMMDD
*
* Category rank dominates the comparison (it's multiplied by a
* number bigger than any possible YYYYMMDD value), so category
* always wins first; the date only breaks ties within the same
* category.
*
* -----
* Commands (one per line of the test file)
* -----
* EMAIL <sender category>,<subject line>,<date>
*     -> insert a new email into the heap
*
* NEXT
*     -> PEEK at (do not remove) the highest-priority email
*         and print Sender / Subject / Date. Calling NEXT
*         twice in a row with no READ in between prints the
*         same email both times, since nothing was removed.
*
* READ
*     -> POP the highest-priority email off the heap. Prints
*         nothing (the CEO "dealt with it" silently). Calling
*         READ twice in a row with no NEXT in between simply
*         discards the two highest-priority emails, neither
*         of which is ever displayed.
*
* COUNT
*     -> print how many unread emails remain in the queue

```

```

*
* NEXT/READ on an empty queue is handled gracefully (no crash,
* a friendly message instead of a listing).
*
* -----
* Running
* -----
* ./email_priority < testfile.txt      (read commands via stdin)
* ./email_priority testfile.txt        (read commands from a file
*                                     named on the command line)
* Both invocation styles are supported so the grader can use
* whichever it prefers.
* ===== */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_SENDER_LEN    32
#define MAX_SUBJECT_LEN   256
#define MAX_DATE_LEN      16
#define MAX_LINE_LEN      1024
#define INITIAL_CAPACITY  16

/* -----
* Email record stored in the heap.
* ----- */
typedef struct {
    char sender[MAX_SENDER_LEN];    /* sender category, e.g. "Boss" */
    char subject[MAX_SUBJECT_LEN];  /* subject line (no commas) */
    char date[MAX_DATE_LEN];        /* MM-DD-YYYY, kept as given */
    long priorityKey;               /* combined category+date sort key */
} Email;

/* -----
* MaxHeap: a growable array of Emails plus a logical size.
* ----- */
typedef struct {
    Email *data;
    int size;
    int capacity;

```

```

} MaxHeap;

/* =====
 * Heap helper / core functions -- all written from scratch.
 * ===== */

/* Allocate the heap's backing array. */
void heapInit(MaxHeap *h) {
    h->capacity = INITIAL_CAPACITY;
    h->size = 0;
    h->data = (Email *)malloc(sizeof(Email) * (size_t)h->capacity);
    if (h->data == NULL) {
        fprintf(stderr, "Fatal: memory allocation failed in heapInit.\n");
        exit(1);
    }
}

/* Release everything the heap owns. */
void heapFree(MaxHeap *h) {
    free(h->data);
    h->data = NULL;
    h->size = 0;
    h->capacity = 0;
}

/* Double the backing array's capacity when it fills up. */
void heapGrow(MaxHeap *h) {
    int newCapacity = h->capacity * 2;
    Email *newData = (Email *)realloc(h->data, sizeof(Email) * (size_t)newCapacity);
    if (newData == NULL) {
        fprintf(stderr, "Fatal: memory allocation failed in heapGrow.\n");
        exit(1);
    }
    h->data = newData;
    h->capacity = newCapacity;
}

void swapEmails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

```

```

/* Restore heap order by moving the element at 'index' upward
 * toward the root while it is bigger than its parent. */
void siftUp(MaxHeap *h, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (h->data[index].priorityKey > h->data[parent].priorityKey) {
            swapEmails(&h->data[index], &h->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

/* Restore heap order by moving the element at 'index' downward,
 * always swapping with the larger of its two children, until
 * both children (if any) are smaller or it has none. */
void siftDown(MaxHeap *h, int index) {
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < h->size && h->data[left].priorityKey > h->data[largest].priorityKey)
        {
            largest = left;
        }
        if (right < h->size && h->data[right].priorityKey >
h->data[largest].priorityKey) {
            largest = right;
        }
        if (largest == index) {
            break;
        }
        swapEmails(&h->data[index], &h->data[largest]);
        index = largest;
    }
}

/* Insert a new email into the heap, keeping heap order. */
void heapInsert(MaxHeap *h, Email e) {

```

```

    if (h->size == h->capacity) {
        heapGrow(h);
    }
    h->data[h->size] = e;
    siftUp(h, h->size);
    h->size++;
}

/* Remove and return the highest-priority email.
 * Returns 1 and fills *out on success, 0 if the heap is empty. */
int heapExtractMax(MaxHeap *h, Email *out) {
    if (h->size == 0) {
        return 0;
    }
    *out = h->data[0];
    h->size--;
    h->data[0] = h->data[h->size];
    siftDown(h, 0);
    return 1;
}

/* Look at the highest-priority email without removing it.
 * Returns 1 and fills *out on success, 0 if the heap is empty. */
int heapPeek(MaxHeap *h, Email *out) {
    if (h->size == 0) {
        return 0;
    }
    *out = h->data[0];
    return 1;
}

/* =====
 * Parsing helpers
 * ===== */

/* Trim leading/trailing whitespace from a string, in place. */
void trim(char *str) {
    int start = 0;
    while (isspace((unsigned char)str[start])) {
        start++;
    }
    if (start > 0) {

```

```

        memmove(str, str + start, strlen(str) - (size_t)start + 1);
    }
    int len = (int)strlen(str);
    while (len > 0 && isspace((unsigned char)str[len - 1])) {
        str[len - 1] = '\0';
        len--;
    }
}

/* Map a sender category string to its priority rank.
 * Higher number = higher priority = read sooner. */
int categoryRank(const char *category) {
    if (strcmp(category, "Boss") == 0)        return 5;
    if (strcmp(category, "Subordinate") == 0)  return 4;
    if (strcmp(category, "Peer") == 0)         return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0)  return 1;
    return 0; /* unrecognized category: treated as lowest priority */
}

/* Convert MM-DD-YYYY into a comparable integer YYYYMMDD.
 * Bigger result = later (more recent) date. */
long dateToInt(const char *date) {
    int mm = 0, dd = 0, yyyy = 0;
    if (sscanf(date, "%d-%d-%d", &mm, &dd, &yyyy) != 3) {
        return 0; /* malformed date sorts as oldest possible */
    }
    return (long)yyyy * 10000L + (long)mm * 100L + (long)dd;
}

/* Parse a line of the form:
 *     EMAIL <sender category>,<subject line>,<date>
 * (commas are the field delimiters; the subject line itself is
 * guaranteed not to contain commas) and insert it into the heap. */
void processEmailCommand(MaxHeap *h, char *line) {
    /* Skip the leading "EMAIL " token. */
    char *rest = line + 6;

    char *senderTok = strtok(rest, ",");
    char *subjectTok = strtok(NULL, ",");
    char *dateTok = strtok(NULL, ",");

```

```

    if (senderTok == NULL || subjectTok == NULL || dateTok == NULL) {
        fprintf(stderr, "Skipping malformed EMAIL line: %s\n", line);
        return;
    }

    Email e;
    strncpy(e.sender, senderTok, MAX_SENDER_LEN - 1);
    e.sender[MAX_SENDER_LEN - 1] = '\0';
    trim(e.sender);

    strncpy(e.subject, subjectTok, MAX_SUBJECT_LEN - 1);
    e.subject[MAX_SUBJECT_LEN - 1] = '\0';
    trim(e.subject);

    strncpy(e.date, dateTok, MAX_DATE_LEN - 1);
    e.date[MAX_DATE_LEN - 1] = '\0';
    trim(e.date);

    int rank = categoryRank(e.sender);
    long dateVal = dateToInt(e.date);
    e.priorityKey = (long)rank * 100000000L + dateVal;

    heapInsert(h, e);
}

/* =====
 * Command handlers
 * ===== */

void handleNext(MaxHeap *h) {
    Email e;
    if (heapPeek(h, &e)) {
        printf("Next email:\n");
        printf("\tSender: %s\n", e.sender);
        printf("\tSubject: %s\n", e.subject);
        printf("\tDate: %s\n", e.date);
        printf("\n");
    } else {
        printf("No emails to read.\n\n");
    }
}

```

```

void handleRead(MaxHeap *h) {
    Email e;
    /* Silently discard the top email. If the heap is empty,
     * heapExtractMax simply returns 0 and nothing happens --
     * no crash, no output, matching the "handle gracefully"
     * requirement. */
    heapExtractMax(h, &e);
}

void handleCount(MaxHeap *h) {
    if (h->size == 1) {
        printf("There is 1 email to read.\n\n");
    } else {
        printf("There are %d emails to read.\n\n", h->size);
    }
}

/* =====
 * Main: read commands one line at a time from a file named on
 * the command line, or from stdin if no filename is given.
 * ===== */

int main(int argc, char *argv[]) {
    FILE *input = stdin;

    if (argc > 1) {
        input = fopen(argv[1], "r");
        if (input == NULL) {
            fprintf(stderr, "Could not open file: %s\n", argv[1]);
            return 1;
        }
    }

    MaxHeap heap;
    heapInit(&heap);

    char line[MAX_LINE_LEN];
    while (fgets(line, sizeof(line), input) != NULL) {
        /* Strip trailing newline / carriage return. */
        line[strcspn(line, "\r\n")] = '\0';

        if (strlen(line) == 0) {
            continue; /* ignore blank lines */

```

```

    }

    if (strncmp(line, "EMAIL ", 6) == 0) {
        processEmailCommand(&heap, line);
    } else if (strcmp(line, "NEXT") == 0) {
        handleNext(&heap);
    } else if (strcmp(line, "READ") == 0) {
        handleRead(&heap);
    } else if (strcmp(line, "COUNT") == 0) {
        handleCount(&heap);
    } else {
        fprintf(stderr, "Skipping unrecognized command: %s\n", line);
    }
}

if (input != stdin) {
    fclose(input);
}

heapFree(&heap);
return 0;
}

```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness: From a general perspective, both the Claude and ChatGPT programs run as expected. They take in the user email file, organize it into a queue, and output text based on the commands. They use the correct tools like MaxHeap, and they use the correct commands. ChatGPT, however, does not remove whitespace around the sender, subject, or date. It also ends the program if it encounters an incorrectly formatted email. Claude handles extra spaces. However, the Claude program will cut subject lines at 256 characters, while ChatGPT will handle 1,024 characters. However, both of those are long for a subject line. Claude also places invalid sender categories at priority zero, instead of rejecting. Additionally, an incorrect date can be placed at priority zero. ChatGPT has a predictable behavior in place for emails that have equal priority in all three categories, while Claude does not.

Execution time: Both programs have the same time complexity for each operation. Inserting an email is $O(\log n)$. The NEXT operation is $O(1)$. READ operation is $O(\log n)$. COUNT operation is $O(1)$. However, overall Claude runs a bit faster. Claude reserves fewer characters than ChatGPT, so it takes less time to move data around. Both programs seem to use the same loop types throughout the program. However, Claude does use a single priority key, rather than a different value for each priority category. This makes comparisons easier and can make Claude run slightly faster.

Maintainability: The Claude program has a lot more commentation than the ChatGPT program. It includes prologue comments, descriptions for blocks of code, and some individual line comments. ChatGPT had very few comments. It simply described a few of the functions or lines. The ChatGPT program tends to use more descriptive variables, like heap, email, and index. In those same places, Claude used variables like h, e, and mm. This makes ChatGPT's code easier to comprehend when reading, and therefore easier to maintain. Further, since ChatGPT keeps priority keys separate, they are easier to adjust than in the Claude Program. Both programs made good use of named constants throughout the program, rather than hardcoding numbers.

Space Complexity: Both programs have the same $O(n)$ complexity. Both grow based on the number of emails in the queue. However, on a per-email basis, Claude's code is more efficient. For the sender array, both programs take in 32 characters. However, for the subject array, ChatGPT takes in 1,024 while Claude takes in 256. This makes Claude's email structure much smaller than ChatGPT's. In terms of variable usage, both programs use about the same amount of variables.

f. Select one to use as the basis for this assignment and justify why you picked it.

I am going to choose to edit the Claude program because it had a more efficient Space complexity structure and execution time efficiency. This sets Claude as a good foundation for edits. Further, I believe that Claude is better in terms of correctness. Technically, it takes in smaller subject lines. But, I do not think that it is necessary to accept subject lines bigger than 256 characters. Additionally, Claude handles whitespace while ChatGPT does not. Finally, Claude included far better commenting than ChatGPT. This will make the editing process easier to understand and approach. One downside is the slightly worse variable naming that occurred in Claude.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: Since emails with the same priority could be added in a random order, I added a feature that added the tied emails to the queue based on arrival order. Additionally, the Claude program had a bug where it would place unrecognized senders at priority 0. I changed the code so that senders must be in one of the five categories. The same issue occurred for misformatted dates. I adjusted the date function logic to handle the issue correctly.

Execution time: To reduce execution time, I changed heapPeek so that it returns a pointer instead of copying the email into a new variable. This will slightly increase the speed of the NEXT function. Additionally, READ stores the email it is removing as a variable. However, that variable is never used. I removed the creation of the variable, which should improve execution time.

Space Complexity: There were not many meaningful changes that could be made to the space complexity of the Claude program. However, I did change the size of the MAX_SENDER_LEN to 16 and MAX_DATE_LEN to 11. This could be done because the longest possible sender is 15 characters and the longest possible date is 10 characters.

Maintainability: To improve maintainability, I added comments to every line of the program and ensured all functions were described. Additionally, I changed variables so that the code was more readable. I made edits like changing “h” to “heap”, “e” to “email”, “mm” to “month”, etc. Further, there was a priority multiplier in the code that was unlabeled. I named it to improve readability.